Reproduce Logistic Model (LASSO)

Justin C. Eldridge

2024-12-07

Logistic Model (LASSO)

This document contains all the code necessary to reproduce the logistic models along with the associated visualizations.

Contents:

1. Variables
2. Lasso Model
3. Data Split
4. Tuning Lasso
5. Logistic Regression Model (Entire Data Set)
6. Logistic Regression Models by Team
7. Data Cleaning
8. Visualizations

1. Variables:

Using our background knowledge and drawing on our EDA we decided to include the following variables in our models.

Game Characteristics:

- **`game_id`**: Unique identifier for the game
- **`play_id`**: Unique identifier for a specific play
- **`possession_team`**: Identifies the offensive team
- **`week`**: Identifies which week of the season the game was in
- **`run_pass`**: Indicates if the play was a run or a pass (response variable)
- **`down`**: Identifies which down the offensive team was on for a particular play
- **`yards_to_go`**: The number of yards needed to get a first down
- **`yards_to_endzone`**: The number of yards to the opponent's endzone
- **`redzone`**: Indicates whether the offense is within 20 yards of the opponent's endzone
- **`sec_in_half`**: The number of seconds remaining on the play clock in the current half
- **`epa_lag`**: Serial correlation of expected points added

Team Characteristics:

- **offense_formation**: Categorical variable indicating what formation the offense uses
- **receiver_alignment**: Enumerated as 0x0, 1x0, 1x1, 2x0, 2x1, 2x2, 3x0, 3x1, 3x2 (text)
- **curr_win_percentage**: Win percentage for the current offensive team
- **form_change**: Uses shifts to determine if the offense changed formation before the ball was snapped

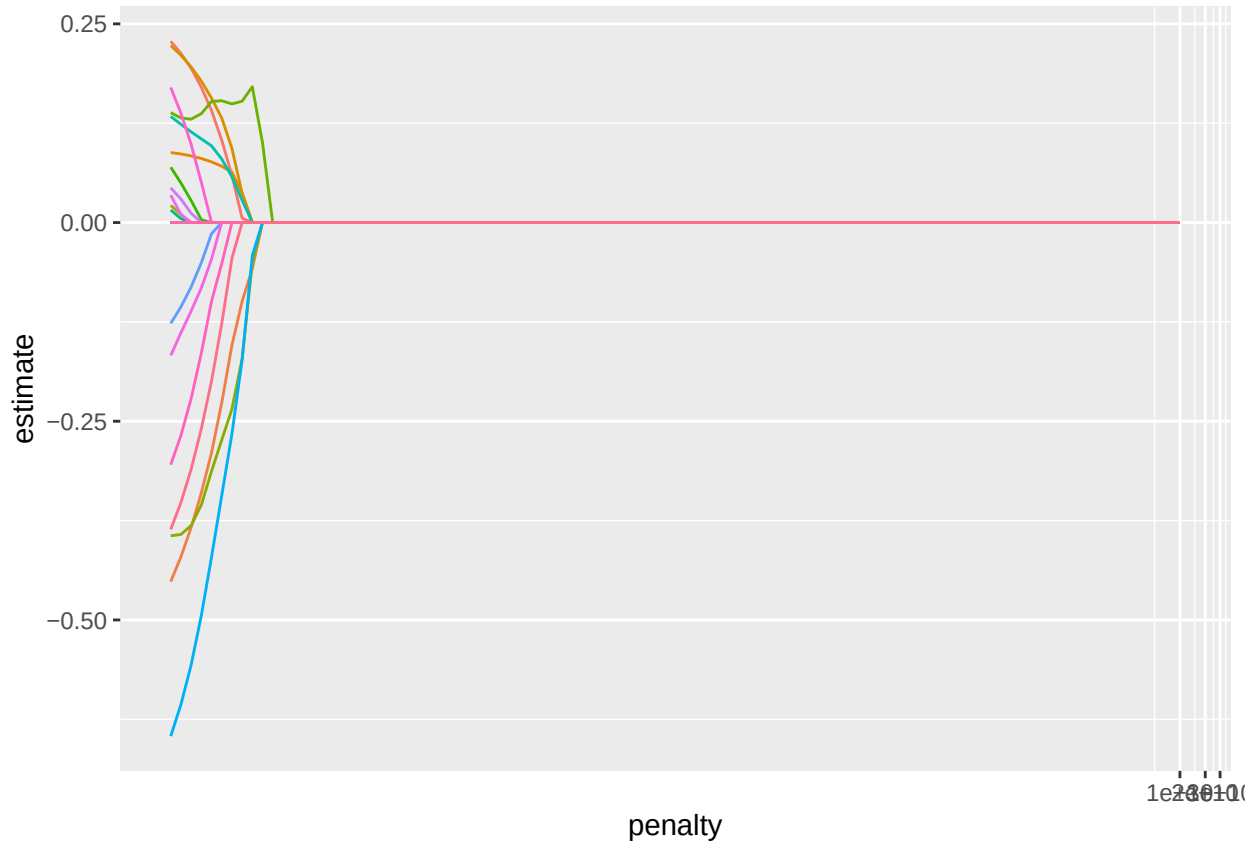
Player Characteristics:

- **wr_motion**: Indicates if a wide receiver moved after the line was set
- **te_motion**: Indicates if a tight end moved after the line was set
- **rb_motion**: Indicates if a running back moved after the line was set
- **wr_atsnap**: Indicates if a wide receiver was in motion at the time of the snap
- **te_atsnap**: Indicates if a tight end was in motion at the time of the snap
- **rb_atsnap**: Indicates if a running back was in motion at the time of the snap

2. LASSO Model:

```r
#Load in the data set
nfl_df <- read.csv("data/modeling_data_frame.csv")

#Make the run pass variable a factor
nfl_df$run_pass <- as.factor(nfl_df$run_pass)

#Convert the boolean columns into numeric ones
bool_cols <- c("redzone", "wr_motion", "te_motion", "rb_motion",
               "wr_atsnap", "te_atsnap", "rb_atsnap")
nfl_df[bool_cols] <- lapply(nfl_df[bool_cols], as.numeric)

#Remove NA values
nfl_df <- nfl_df %>%
  drop_na()

#Use a lasso model to do varaible selection.

#Create a sequence of lambda values to try
lambda <- lambda <- 10^seq(-2, 10, length.out = 100)

#Create a recipe for the lasso model and normalize the predictors.
lasso_rec <- recipe(run_pass~., data=nfl_df) %>%

  #Remove unnecessary variables
  step_rm(game_id, play_id, possession_team,week) %>%

  step_novel(all_nominal_predictors()) %>%

  #Fix error for missing values by assigning them with unknown
  step_unknown(all_nominal_predictors()) %>%

  #Fix error where some variables are categorical and not numeric
  step_dummy(all_nominal_predictors()) %>%
```

```r
  #Fix error from some of them having zero variance
  step_zv(all_predictors())%>%

  #Normalize the numeric predictors to have mean=0 and sd=1
  step_normalize(all_numeric_predictors())

#Create an empty data frame to store our lasso estimates
lasso_ests <- data.frame()

#Fit a lasso model for each lambda in our grid
for(lam in lambda){
  lasso_spec <- logistic_reg(mixture=1, penalty=lam)%>%
    set_mode("classification") %>%
    set_engine("glmnet")

  #Fit the model
  workflow() %>%
    add_model(lasso_spec) %>%
    add_recipe(lasso_rec) %>%
    fit(nfl_df) %>%
    tidy() %>%
    bind_rows(lasso_ests)-> lasso_ests
}

  #Create a line plot of
lasso_ests |>
  filter(term != "(Intercept)") |>
  ggplot() +
  geom_line(aes(penalty, estimate, group = term, colour = term)) +
  coord_trans(x = "log10")+
  #The legend is so large it obstructs the graph. You can comment out this line
  # and view the graph in a new window to get a better idea.
  theme(legend.position="none")
```



3. Data Split:

```
#Create the training and testing sets of data
  #Use the first 6 weeks of data for training
  #Use the last 3 weeks of dat for testing

#Grab the data
nfl_lasso_df <- nfl_df

#Make sure that run pass is a factor not a character type
nfl_lasso_df$run_pass <- as.factor(nfl_lasso_df$run_pass)

#Encode the boolean columns as numeric
nfl_lasso_df[bool_cols] <- lapply(nfl_lasso_df[bool_cols], as.numeric)

#Create the training set
nfl_train <- nfl_lasso_df %>%
  filter(week==c(1,2,3,4,5,6))

#Create the test set
nfl_test <- nfl_lasso_df %>%
  filter(week==c(7,8,9))
```

4. Tuning Lasso:

```
#Fit a lasso model and determine the optimum lambda for varaible selection

#Create a data frame for tuning lambda
tune_df <- data.frame(lambda=lambda)

#Perform 10-fold cross-validation to get an estimate of test MSE for each
#labmda in our grid
nfl_10fld_cv <- vfold_cv(nfl_df, v=10)

#Specify the model
lasso_spec <- logistic_reg(mixture = 1, penalty = tune("lambda")) %>%
  set_mode("classification") %>%
  set_engine("glmnet")

#Create the workflow object for lasso
lasso_workflow <-workflow() %>%
  add_model(lasso_spec) %>%
  add_recipe(lasso_rec)

#Perform the cross validation
tune_results <- tune_grid( lasso_workflow,
                           resamples=nfl_10fld_cv,
                           #For some reason it works if I let the labmda values
                           #be created automatically but not when I try to use
                           #the labmda values in the tuning
                           grid=50)

#Determine the best lambda value
best_lambda <- select_best(tune_results, metric="accuracy")

#Create the lasso model with the optimum lambda
final_lasso_workflow <- finalize_workflow(lasso_workflow,  best_lambda)

final_lasso_model <- fit(final_lasso_workflow, data=nfl_train) -> final_lasso_fit

final_model <- extract_fit_parsnip(final_lasso_model)

#Convert the coefficients to a tidy data frame
selected_variables <- tidy(final_model)

#Generate predictions on the test dataset
lasso_predictions <- predict(final_lasso_model, nfl_test, type = "class") %>%
  bind_cols(nfl_test)

#Create a confusion matrix
confusion_mat <- conf_mat(
  data = lasso_predictions,
  truth = run_pass,
  estimate = .pred_class
)
```

```r
#Print the confusion matrix
print(confusion_mat)
```

```
##           Truth
## Prediction pass run
##       pass  740 192
##       run   200 483
```

5. Logistic Regression Model (Entire Data Set):

```r
#Fit the logistic model on the entire data set using the varaibles from the
#lasso model.

#Look at the output from the optimal lasso model to do variable selection
selected_vars <-c("down", "yards_to_go", "yards_to_endzone", "sec_in_half",
                  "epa_lag", "curr_win_percentage", "form_change", "wr_motion",
                  "te_motion","rb_motion","wr_atsnap", "te_atsnap",
                  "offense_formation", "receiver_alignment")

#Drop the few data points that are in the testing set but not the training set
nfl_train <- nfl_train %>% filter(offense_formation %in% nfl_test$offense_formation,
                                    receiver_alignment %in% nfl_test$receiver_alignment)

nfl_test <- nfl_test %>% filter(offense_formation %in% nfl_train$offense_formation,
                                  receiver_alignment %in% nfl_train$receiver_alignment)

#Create a formula for logistic regression using selected variables
logistic_formula <- as.formula(paste("run_pass ~",
                                     paste(selected_vars, collapse = " + ")))

#Fit the logistic regression model using the selected variables
final_logistic_model <- glm(
  formula = logistic_formula,
  data = nfl_train,
  family = binomial()
)

# Generate predictions on the test set
logistic_predictions_prob <- predict(final_logistic_model,
                                     newdata = nfl_test, type = "response")

#Convert probabilities to class labels based on a threshold of 0.5
logistic_predictions_class <- ifelse(logistic_predictions_prob > 0.5, "run", "pass")

#Add predictions to the test dataset
nfl_test <- nfl_test %>%
  mutate(predicted_run_pass = logistic_predictions_class)

#Create a confusion matrix
confusion_matrix <- table(
  Truth = nfl_test$run_pass,
```

```r
  Prediction = nfl_test$predicted_run_pass
)

#Make sure that the precited pass column is also a factor like the truth col
nfl_test$predicted_run_pass <- as.factor(nfl_test$predicted_run_pass)

#Create a confusion matrix with the accuracy
confusionMatrix(nfl_test$predicted_run_pass, nfl_test$run_pass)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction pass run
##       pass  736 184
##       run   203 491
##
##                Accuracy : 0.7602
##                  95% CI : (0.7386, 0.7809)
##     No Information Rate : 0.5818
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.5092
##
##  Mcnemar's Test P-Value : 0.3602
##
##             Sensitivity : 0.7838
##             Specificity : 0.7274
##          Pos Pred Value : 0.8000
##          Neg Pred Value : 0.7075
##              Prevalence : 0.5818
##          Detection Rate : 0.4560
##    Detection Prevalence : 0.5700
##       Balanced Accuracy : 0.7556
##
##        'Positive' Class : pass
##
```

6. Logistic Regression Models by Team:

```r
#Fit the optimal logistic regression model for each team


#Create a list to store the models for each team as well as the confusion
#matrices
model_info <- list("nfl_team" = unique(nfl_df$possession_team),
                   "train_fit" = as.list(seq(32)),
                   "prediction" = as.list(seq(32)),
                   "conf_mat" = as.list(seq(32)),
                   "m_corr"= as.list(seq(32)))

#Specify the logistic formula using the variables selected from lasso
```

```r
logistic_formula <- as.formula(paste("run_pass ~",
                                     paste(selected_vars, collapse = " + ")))

#Use a for loop to create a model for each team and store it in the model info
#list
for (i in c(1:32)){

  #Grab the current team
  team<-model_info$nfl_team[i]

  #Add a print statement so we can keep track while its running
  print(paste("Working on team", team))

  #Create a data frame with the data for the current team
  team_df <- nfl_df %>%
    filter(possession_team==team) %>%
    mutate(run_pass = as.factor(run_pass))

  #troublesome receiver alignment value for denver remove it
  if(team=="DEN"){
    team_df <- team_df %>%
      filter(receiver_alignment !="2x0")
  }

  #Sort the data by week
  weeks <- sort(unique(team_df$week))

  #Split the data into training and testing
  team_train <- team_df %>%
    filter(week==c(1,2,3,4,5,6))

  team_test <- team_df %>%
    filter(week==c(7,8,9))

  #Drop the few data points that are in the testing set but not the training set
  team_train <- team_train %>%
    filter(offense_formation %in% team_test$offense_formation,
           receiver_alignment %in% team_test$receiver_alignment)

  team_test <- team_test %>%
    filter(offense_formation %in% team_train$offense_formation,
           receiver_alignment %in% team_train$receiver_alignment)

  #Fit the logistic regression model using the selected variables
  final_logistic_model <- glm(
    formula = logistic_formula,
    data = nfl_train,
    family = binomial()
    )

  #Extract the model
  train_fit <- final_logistic_model$model
```

```r
  #Add the training fit to the list
  model_info$train_fit[[i]] <- train_fit

  # Generate predictions on the test set
  logistic_predictions_prob <- predict(final_logistic_model,
                                       newdata = team_test, type = "response")

  #Convert probabilities to class labels based on a threshold of 0.5
  logistic_predictions_class <- ifelse(logistic_predictions_prob > 0.5,
                                       "run", "pass")

  #Add the predicted classes to the model info list
  model_info$predictions[[i]]<- logistic_predictions_class

  #Add predictions to the test dataset
  team_test <- team_test %>%
    mutate(predicted_run_pass = logistic_predictions_class)

  #Make sure that the precited pass column is also a factor like the truth col
  team_test$predicted_run_pass <- as.factor(team_test$predicted_run_pass)

  #Add the confusion matrix to the model info list
  model_info$conf_mat[[i]]<-confusionMatrix(team_test$predicted_run_pass,
                                            team_test$run_pass)

  #Calculate Matthews correlation coefficient (mltools)
  model_info$m_corr[[i]]<- mcc(preds=team_test$predicted_run_pass,
                               actuals= team_test$run_pass)

  #Add print statements to view the model progress

  #Print the first couple of confusion matrices as an example
  if(i<=2){
    print(confusionMatrix(team_test$predicted_run_pass,team_test$run_pass))
    print(model_info$m_corr[[i]])
  }

}
```

```
## [1] "Working on team BUF"

## Confusion Matrix and Statistics
##
##           Reference
## Prediction pass run
##       pass   12   4
##       run     6   7
##
##                Accuracy : 0.6552
##                  95% CI : (0.4567, 0.8206)
##     No Information Rate : 0.6207
##     P-Value [Acc > NIR] : 0.4303
##
```

```
##                   Kappa : 0.2927
##
##  Mcnemar's Test P-Value : 0.7518
##
##             Sensitivity : 0.6667
##             Specificity : 0.6364
##          Pos Pred Value : 0.7500
##          Neg Pred Value : 0.5385
##              Prevalence : 0.6207
##          Detection Rate : 0.4138
##    Detection Prevalence : 0.5517
##       Balanced Accuracy : 0.6515
##
##        'Positive' Class : pass
##
## [1] 0.2956562
## [1] "Working on team LA"

## Confusion Matrix and Statistics
##
##           Reference
## Prediction pass run
##       pass   11   5
##       run    12   7
##
##                Accuracy : 0.5143
##                  95% CI : (0.3399, 0.6862)
##     No Information Rate : 0.6571
##     P-Value [Acc > NIR] : 0.9727
##
##                   Kappa : 0.0541
##
##  Mcnemar's Test P-Value : 0.1456
##
##             Sensitivity : 0.4783
##             Specificity : 0.5833
##          Pos Pred Value : 0.6875
##          Neg Pred Value : 0.3684
##              Prevalence : 0.6571
##          Detection Rate : 0.3143
##    Detection Prevalence : 0.4571
##       Balanced Accuracy : 0.5308
##
##        'Positive' Class : pass
##
## [1] 0.05868912
## [1] "Working on team ATL"
## [1] "Working on team NO"

## [1] "Working on team CAR"

## [1] "Working on team CLE"

## [1] "Working on team CHI"
```

```
## [1] "Working on team SF"

## [1] "Working on team PIT"

## [1] "Working on team CIN"

## [1] "Working on team DET"

## [1] "Working on team PHI"

## [1] "Working on team IND"

## [1] "Working on team HOU"

## [1] "Working on team NE"

## [1] "Working on team MIA"

## [1] "Working on team NYJ"

## [1] "Working on team BAL"
## [1] "Working on team NYG"
## [1] "Working on team TEN"

## [1] "Working on team JAX"

## [1] "Working on team WAS"

## [1] "Working on team KC"

## [1] "Working on team ARI"
## [1] "Working on team LAC"

## [1] "Working on team LV"

## [1] "Working on team MIN"

## [1] "Working on team GB"
## [1] "Working on team DAL"

## [1] "Working on team TB"

## [1] "Working on team SEA"

## [1] "Working on team DEN"
```

```r
#Add an ending print statement
print("Complete")
```

```
## [1] "Complete"
```

```r
#Save the data
saveRDS(model_info, "Logistic Model Info by Team.rds")
```

7. Data Cleaning:

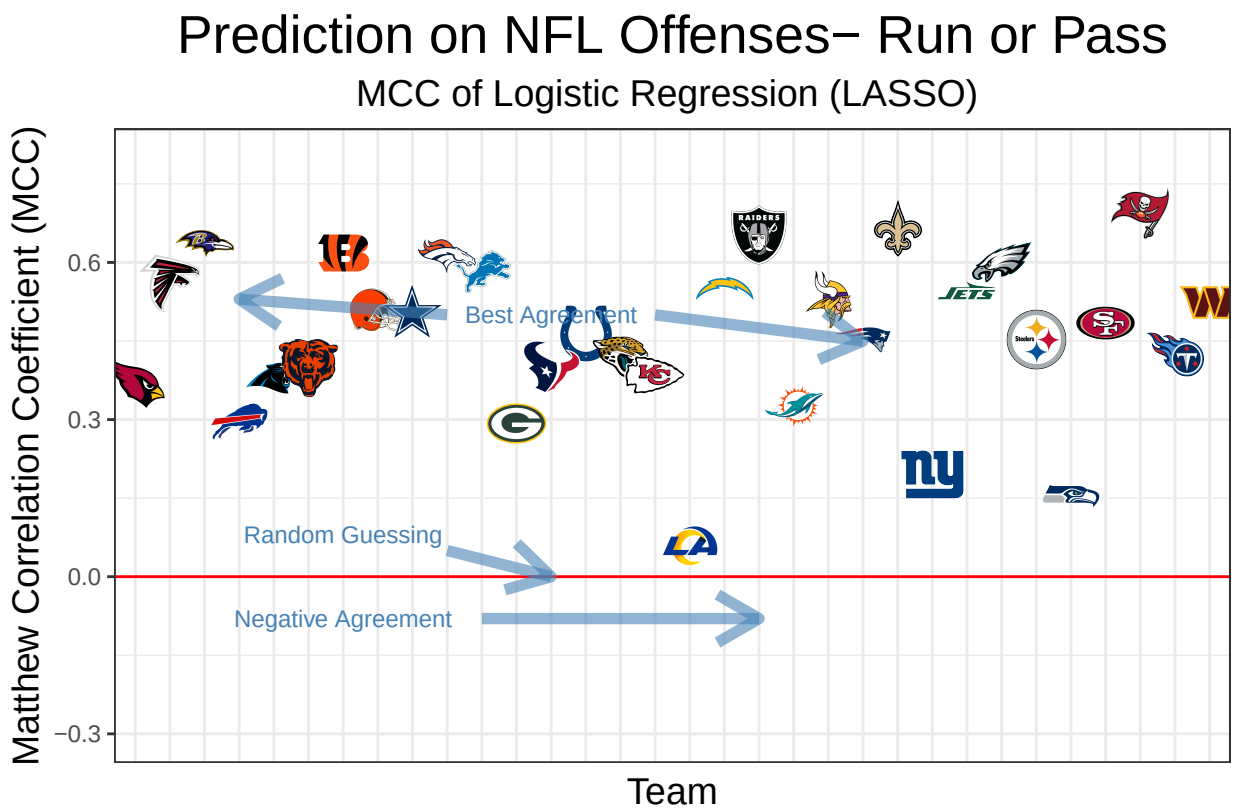
```r
#Load in the data
log_model_info <- readRDS("Logistic Model Info by Team.rds")

#Convert some of the list objects into data frames for plotting
team_names <- as.data.frame(log_model_info$nfl_team)

#Rename the column
team_names <- team_names %>%
  rename(nfl_team = `log_model_info$nfl_team`)

mcc_vals <- as.data.frame(log_model_info$m_corr)

mcc_vals_long <- mcc_vals %>%
  pivot_longer(
    cols = everything(),
    names_to = "variable",
    values_to = "mcc_value"
  )

#Combine the two into a single data frame for plotting
plot_df <- data.frame(Team = team_names$nfl_team,
                      MCC = mcc_vals_long$mcc_value)


#Use Chandlers code to create the appropriate data for plotting

#Create a data frame with indicators for pass and run as well as
#the Mcnemar's Test P-Value, accuracy p-value, sensitivity, specificity,etc
log_data <- data.frame(team=unique(log_model_info$nfl_team),
                           pass_1= -1, pass_0= -1, run_1=-1, run_0=-1,
                           mn_pvalue=1, acc_pvalue=1, spec=0, sens=0,
                           pos_pred=0, neg_pred=0, pos_class="")

#Calculate the values that belong in the new columns

for(i in c(1:32)){

  #Extract the current team
  team<- log_data$team[i]

  log_data$pass_1[i] <- log_model_info$conf_mat[[match(team,log_model_info$nfl_team)]]$table[1,1]
```

```
  log_data$pass_0[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$table[2,1]
  log_data$run_1[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$table[2,2]
  log_data$run_0[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$table[1,2]
  log_data$mn_pvalue[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$overall[7]
  log_data$acc_pvalue[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$overall[6]
  log_data$sens[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$byClass[1]
  log_data$spec[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$byClass[2]
  log_data$pos_pred[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$byClass[3]
  log_data$neg_pred[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$byClass[4]
  log_data$pos_class[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$positive
}
```

8. Visualizations:

```
#Use the correlations in the model info list to plot correllation by team
corr_plot <- plot_df %>%
  ggplot(aes(x=Team, y=MCC))+

  #Add the team logos for clarity
  geom_nfl_logos(aes(team_abbr = Team,
                     width = 0.075,
                     height=0.1))+

  #Add a horizontal line at y=0 to visualize the boundary
  geom_hline(yintercept = 0, color="red")+

  #Label the two points with the highest (best) MCC
  annotate("text", x="HOU", y=0.5, label="Best Agreement",
           color="steelblue", size=3)+
  annotate("segment",
           x="DEN", xend="BUF",
           y=0.5, yend=0.53,
           size=2,colour="steelblue",
           alpha=0.6,arrow=arrow())+
  annotate("segment",
           x="KC", xend="NE",
           y=0.5, yend=0.45,
           size=2,colour="steelblue",
           alpha=0.6,arrow=arrow())+

  #Label a point where MCC is approximately 0
  #i.e. teams where the model was as good as randomly guessing
  annotate("text", x="CIN", y=0.08, label="Random Guessing",
           color="steelblue", size= 3)+
  annotate("segment", x="DEN" ,xend="HOU", y=0.05, yend=0,
           size=2, alpha=0.6,color="steelblue",
           arrow=arrow())+

  #Label a point with negative agreement
  annotate("text", x="CIN", y=-0.08, label="Negative Agreement",
           color="steelblue", size=3)+
  annotate("segment", x="DET" ,xend="LV", y=-0.08, yend=-0.08,
```

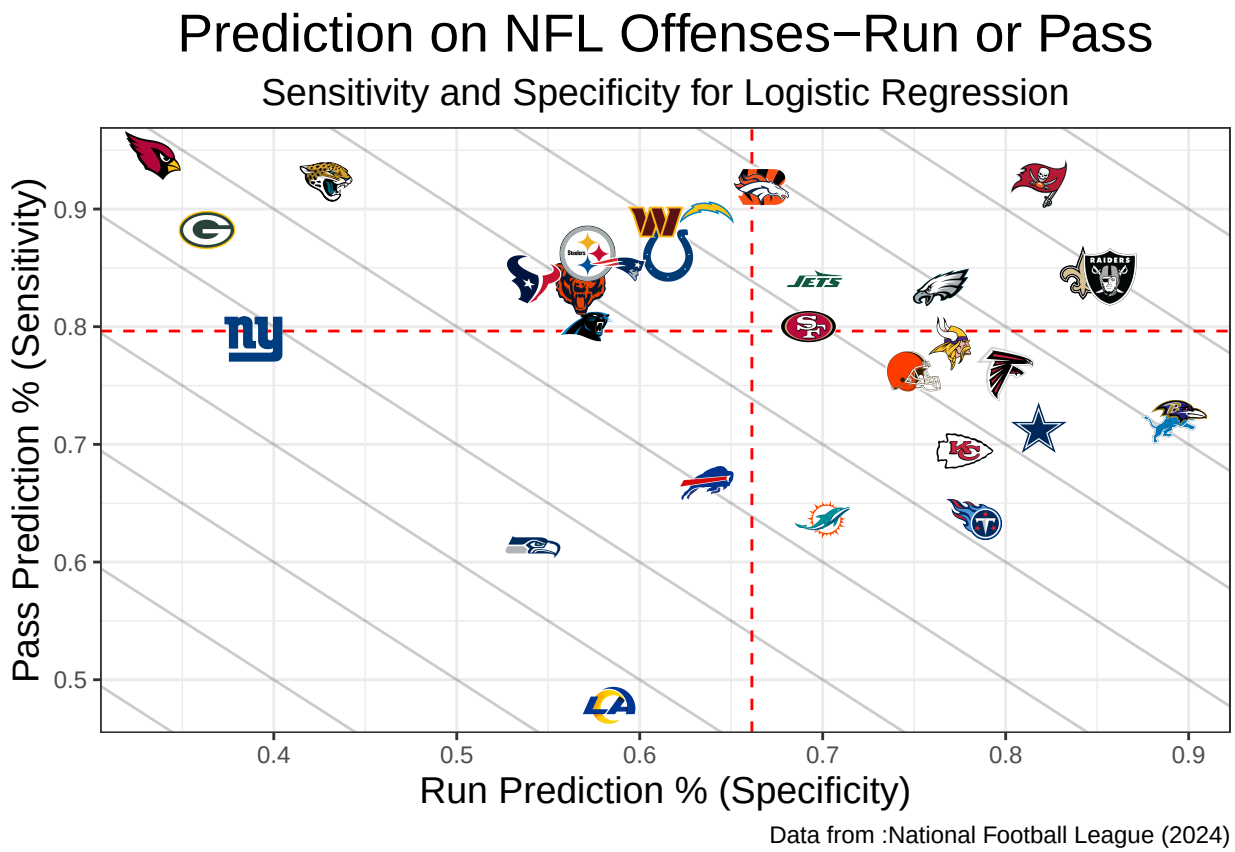
```r
          size=2, alpha=0.6,color="steelblue",
          arrow=arrow())+

  #Change the y-axis limits to be consistent with the other models MCC
  scale_y_continuous(limits=c(-0.3, 0.8))+

  #Add appropriate labels
  labs(x="Team",
       y="Matthew Correlation Coefficient (MCC)",
       title="Prediction on NFL Offenses- Run or Pass",
       subtitle="MCC of Logistic Regression (LASSO) ",
       caption="Data from: NFL Big Data Bowl (2025)")+

  theme_bw()+

  #Adjust plot elements for neatness
  theme(plot.title= element_text(hjust=0.5, size=20),
        plot.subtitle = element_text(hjust = 0.5, size=14),

        #Remove the x-axis labels and ticks to clean up the plot
        axis.text.x=element_blank(),
        axis.ticks.x=element_blank(),
        axis.title.y= element_text(size=14),
        axis.title.x= element_text(size=14))

#Display the plot
corr_plot
```



Data from: NFL Big Data Bowl (2025)

```
#Save the plot and put it into the graphs folder
ggsave("graphs/MCC_Logistic.png", plot=corr_plot,
       width=12, height=7)

#Save the plot in a different size for the paper
ggsave("graphs/Images for paper/MCC_log.png", plot= corr_plot,
       height=4, width=6.5)


#Create a plot of specificity vs. sensitivity
sens_vs_spec_plot <-log_data %>%
  ggplot(aes(x = spec, y = sens)) +
  # draw the background lines. sequence for intercept will need changing
  ggplot2::geom_abline(slope = -1, intercept = seq(2, 0.6, -0.1), alpha = .2) +
  # draw the red dashed mean lines for the data
  nflplotR::geom_mean_lines(aes(x0 = spec , y0 = sens)) +

  #Add the team logos for clarity
  geom_nfl_logos(aes(team_abbr = team,
                     width = 0.075,
                     height=0.1))+

  #Add appropriate labels
  labs(x="Run Prediction % (Specificity)",
       y="Pass Prediction % (Sensitivity)",
       subtitle="Sensitivity and Specificity for Logistic Regression",
```

```r
       title="Prediction on NFL Offenses-Run or Pass",

       #Cite the data source
       caption="Data from :National Football League (2024)")+
  theme_bw()+

  #Justify plot elements for neatness
  theme(plot.title= element_text(hjust=0.5, size=20),
        plot.subtitle = element_text(hjust=0.5, size=14),
        axis.title.x= element_text(size=14),
        axis.title.y= element_text(size=14)
        )

#Display the plot
sens_vs_spec_plot
```




```r
#Save the plot with a width=12, height =7
ggsave("graphs/Sens_vs_spec_Logistic.png", plot=sens_vs_spec_plot,
       width=12, height=7)

#Save the plot in a different size for the paper
ggsave("graphs/Images for paper/Sens_vs_Spec_Log.png", plot=sens_vs_spec_plot,
       height=4, width=6.5)
```